

Decision Structures, Loops & Functions

Miss Shumaila

FAST | Semester 2

What is a Decision Structure?

- A decision structure allows a program to choose between different paths of execution.
- It evaluates a condition that results in either True or False.
- Based on the result, a specific block of code executes.
- In real life:
 - If marks are greater than or equal to 50 → student passes.
 - Otherwise → student fails.

Simple if Statement – Full Explanation

- **Syntax:**

```
if condition:  
    statement
```

- **Example Program:**

```
age = 20  
if age >= 18:  
    print('Eligible to vote')
```

Output: Eligible to vote

Explanation Step-by-Step:

1. Variable age stores value 20.
2. Condition (20 >= 18) is evaluated.
3. Condition becomes True.
4. Because it is True, the indented block executes.

if / else Statement – Full Explanation

Syntax:

```
if condition:  
    block1  
else:  
    block2
```

Example Program:

```
marks = 40  
if marks >= 50:  
    print('Pass')  
else:  
    print('Fail')
```

Output: Fail

Explanation:

40 >= 50 is False.

Therefore, the else block executes.

if / elif / else – Multiple Conditions

- Used when there are multiple possible outcomes.
- Python checks conditions from top to bottom.
- Once a True condition is found, remaining conditions are ignored.

Program:

```
marks = 75
if marks >= 85:
    print('Grade A')
elif marks >= 70:
    print('Grade B')
else:
    print('Grade C')
```

Output: Grade B

Logical Operators – Detailed Explanation

- Logical operators combine multiple conditions.
- AND → Both conditions must be True.
- OR → At least one condition must be True.
- NOT → Reverses the condition.

Example:

```
gpa = 3.7
attendance = 85
if gpa >= 3.5 and attendance >= 80:
    print('Scholarship Approved')
```

Output: Scholarship Approved

Nested if Statement – Full Breakdown

- A nested if is an if statement inside another if.
- It is used when one decision depends on another.

Program:

```
registered = True
fees_paid = False
if registered:
    if fees_paid:
        print('Enrollment Confirmed')
    else:
        print('Please pay fees')
```

Output: Please pay fees

Explanation:

- Outer condition is True, so inner condition is checked.
- Inner condition is False, so inner else executes.

What is a Loop?

- A loop is used to repeat a block of code multiple times.
- There are two main types:
 - 1. While Loop
 - 2. For Loop

While Loop

- Used when iterations unknown
- Requires condition update
- Risk: Infinite loop if not updated

While Loop – Complete Explanation

Syntax:

```
while condition:  
    statement
```

Program:

```
count = 1  
while count <= 3:  
    print(count)  
    count += 1
```

Output:

```
1  
2  
3
```

Explanation:

- The loop runs while the condition remains True.
- count increases each time to avoid infinite loop.

For Loop

- Used when iteration count known
- Common with `range()`
- Used with lists, sets, dictionaries

For Loop – Complete Explanation

- Used when the number of repetitions is known.

Program:

```
for i in range(3):  
    print(i)
```

Output:

```
0  
1  
2
```

Explanation:

- `range(3)` generates 0, 1, 2.
- The loop runs exactly three times.

Nested Loops

- Loop inside another loop
- Common for matrices & patterns
- Outer loop controls rows, inner loop controls columns

Nested Loops – Detailed Explanation

- A nested loop is a loop inside another loop.
- The outer loop controls major iterations.
- The inner loop runs completely for each outer iteration.

Program:

```
for i in range(3):  
    for j in range(i+1):  
        print('*', end='')  
    print()
```

Output:

```
*  
**  
***
```

What is a Function?

- A function is a reusable block of code that performs a specific task.
- It helps in modular programming.
- Instead of writing the same code again and again, we create a function once and call it whenever needed.
- It improves readability and reduces repetition.

Why Do We Use Functions?

- 1. Avoid repetition of code
- 2. Make code easier to understand
- 3. Improve maintainability
- 4. Break large problems into smaller parts
- 5. Enable teamwork in large projects

Basic Syntax of a Function

- In Python, a function is defined using the keyword 'def'.
- Syntax:

```
def greet():  
    print('Hello Students!')  
greet()
```
- The indented block is the body of the function.

Function with Parameters

- Parameters allow us to pass values into a function.
- Syntax:
 - `def function_name(parameter):`
 - `statements`
- Program:
 - `def greet(name):`
 - `print('Hello', name)`
 - `greet('Ali')`
- Output:
 - Hello Ali

Function with Return Value

- A function can return a value using the 'return' keyword.
- Return sends the result back to where the function was called.

Function Syntax & Example

Syntax:

```
def function_name(parameters):  
    statements  
    return value
```

Program:

```
def add(a, b):  
    return a + b  
  
result = add(5, 3)  
print(result)
```

Output: 8

Explanation of Return

- • 'a' and 'b' are parameters.
- • 'return a + b' sends the result back.
- • The result is stored in a variable and printed.

Difference: Print vs Return

- `print()` → Displays output on screen.
- `return` → Sends value back to caller.
- Return makes functions reusable in larger programs.

Designing Good Functions

- 1. Function should perform one specific task.
- 2. Use meaningful names.
- 3. Avoid very long functions.
- 4. Add comments if necessary.
- 5. Keep inputs and outputs clear.

Function with Condition – Even/Odd Checker

Program:

```
def check_even_odd(num):  
    if num % 2 == 0:  
        return 'Even'  
    else:  
        return 'Odd'  
  
print(check_even_odd(4))
```

Output: Even

Explanation:

The modulus operator checks remainder.

Business Case – Payroll System Function

Program:

```
def calculate_salary(hours, rate):  
    return hours * rate
```

```
salary = calculate_salary(40, 500)  
print(salary)
```

Output: 20000

Explanation:

Function calculates salary using parameters.

Practice Question 1

- Write a program that checks whether a number is positive or negative.

Solution – Practice Question 1

```
num = -5  
if num > 0:  
    print('Positive')  
else:  
    print('Negative')
```

Output: Negative

Practice Question 2

- Create a function that returns the maximum of two numbers.

Solution – Practice Question 2

```
def find_max(a, b):  
    if a > b:  
        return a  
    else:  
        return b  
  
print(find_max(10, 20))
```

Output: 20